

Basic Python Interview Guide

Detailed Basic Python notes for interview preparation and hands-on practice

Detailed interview preparation and practice notes. Text only. No tables or images.

Use this PDF as source data for your RAG chatbot and as revision material before interviews.

1. Python introduction, features, and use cases

Concept

Python is a high-level, interpreted, dynamically typed language used in scripting, automation, data science, AI, web development, APIs, testing, and backend services. It focuses on readability and developer productivity.

Important details

Important features are simple syntax, rich standard library, platform independence, automatic memory management, huge package ecosystem, and support for procedural, object-oriented, and functional styles. Python code is executed by the Python interpreter. Source code is first compiled to bytecode and then executed by the Python virtual machine.

Common mistakes

Do not say Python is only interpreted in a very simplified way during interviews; mention bytecode execution if asked deeply. Do not confuse dynamically typed with weakly typed. Python is dynamically typed but strongly typed.

Interview answer style

A good answer: Python is popular because it is easy to read, fast to develop with, has many libraries, and is suitable for prototypes as well as production systems. Examples include FastAPI APIs, automation scripts, ML pipelines, and RAG applications.

Small code example

```
print("Python is ready")  
print(type(10))
```

Interview and practice focus

Be ready to explain python introduction, features, and use cases in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

2. Python installation, virtual environment, and project setup

Concept

A Python project should use a virtual environment so each project has its own dependencies. This avoids package version conflicts between projects.

Important details

Common files are app.py or main.py, src folder for modules, data folder for input files, requirements.txt for dependencies, .env for secrets, and README.md for documentation. In deployed apps, secrets should be configured in platform environment variables, not hardcoded.

Common mistakes

Do not commit .venv, __pycache__, API keys, or large generated folders unless required. Do not write pip install commands inside requirements.txt; only package names and versions belong there.

Interview answer style

Explain that you create a venv, activate it, install requirements, run application entry point, and maintain dependencies using requirements.txt.

Small code example

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
python app.py
```

Interview and practice focus

Be ready to explain python installation, virtual environment, and project setup in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

3. Variables, naming rules, and assignment

Concept

Variables are names that refer to objects in memory. Python variables do not store the type themselves; the object has the type. A variable can later refer to a different type.

Important details

Use snake_case for variable names, UPPER_CASE for constants, and meaningful names. Multiple assignment and unpacking are supported. Assignment does not copy mutable objects; it creates another reference unless copied explicitly.

Common mistakes

A common mistake is thinking `b = a` creates a deep copy for lists. For mutable objects, both names can refer to the same list. Use `copy` or `deepcopy` when needed.

Interview answer style

A strong interview answer should mention object references, dynamic typing, and readable naming conventions.

Small code example

```
x = 10
y = x
items = [1, 2]
other = items
other.append(3)
print(items)
```

Interview and practice focus

Be ready to explain variables, naming rules, and assignment in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

4. Numbers, booleans, and None

Concept

Python has int, float, complex, bool, and NoneType. Integers can grow arbitrarily large. Floats are approximate because they are based on binary floating-point representation. None represents absence of value.

Important details

Boolean values are True and False. In conditions, empty collections, zero, None, and empty strings are treated as false-like. Use is None or is not None for None checks.

Common mistakes

Avoid comparing floats directly when precision matters. Avoid writing if x == None; use if x is None. Do not assume empty list and None mean the same thing.

Interview answer style

Interview-ready response: None is a singleton used to represent no value. is checks identity, while == checks value equality.

Small code example

```
value = None
if value is None:
    print("missing")
print(bool([]), bool("text"))
```

Interview and practice focus

Be ready to explain numbers, booleans, and none in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

5. Strings and text processing

Concept

Strings are immutable sequences of Unicode characters. They support indexing, slicing, formatting, searching, replacing, splitting, joining, and case conversion.

Important details

For text-heavy projects such as RAG, cleaning strings is very important. Extra spaces, newlines, noisy characters, and mixed casing can reduce retrieval quality. f-strings are recommended for readable formatting.

Common mistakes

Common mistakes include forgetting that string methods return a new string, not modifying the original string. Another mistake is using + repeatedly in a loop for large text; join is better.

Interview answer style

Explain immutability, common methods, f-strings, and a real use case such as cleaning document text before chunking.

Small code example

```
text = " Python Basics "  
clean = " ".join(text.split()).lower()  
print(f"Clean text: {clean}")
```

Interview and practice focus

Be ready to explain strings and text processing in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

6. Lists and list operations

Concept

A list is an ordered, mutable collection. Lists can store mixed data types, but in clean code they usually store related items. Indexing, slicing, append, extend, insert, remove, pop, sort, and reverse are common operations.

Important details

Lists are used for storing documents, chunks, predictions, rows, API results, and intermediate pipeline data. Slicing returns a new list. Sorting modifies in place unless `sorted()` is used.

Common mistakes

Do not use a mutable list as a default argument. Do not modify a list while looping over it unless you know exactly what you are doing.

Interview answer style

Interview answer: list is mutable and ordered; tuple is immutable and ordered; set is unordered and unique; dictionary maps keys to values.

Small code example

```
docs = []
docs.append("pdf text")
docs.extend(["docx text", "txt text"])
print(docs[:2])
```

Interview and practice focus

Be ready to explain lists and list operations in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

7. Tuples, unpacking, and immutability

Concept

A tuple is an ordered, immutable collection. It is useful for fixed data such as coordinates, configuration pairs, or returning multiple values from a function.

Important details

Tuple immutability means the tuple cannot be changed, but if it contains a mutable object, that inner object can still be modified. Python supports unpacking values into variables.

Common mistakes

Do not say tuples are always completely immutable in a deep sense. A tuple containing a list can still have that internal list changed.

Interview answer style

Good answer: tuples are lightweight, ordered, immutable containers and often used when data should not be structurally changed.

Small code example

```
point = (10, 20)
x, y = point
def stats():
    return 5, 10
low, high = stats()
```

Interview and practice focus

Be ready to explain tuples, unpacking, and immutability in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

8. Sets and uniqueness

Concept

A set stores unique values. It is useful for removing duplicates, membership testing, and mathematical operations like union, intersection, difference, and symmetric difference.

Important details

Set membership is usually faster than list membership for large data. Sets are unordered, so do not rely on element position. Only hashable values can be stored in a set.

Common mistakes

Do not try to store lists or dictionaries inside a set because they are unhashable. Do not expect set output order to remain fixed.

Interview answer style

Interview answer should mention uniqueness, fast membership checks, and set operations.

Small code example

```
skills = {"python", "fastapi", "python"}  
print(skills)  
print("python" in skills)
```

Interview and practice focus

Be ready to explain sets and uniqueness in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

9. Dictionaries and JSON-style data

Concept

A dictionary stores key-value pairs. It is one of the most used Python data structures and maps naturally to JSON objects. Keys must be hashable.

Important details

Useful methods include `get`, `keys`, `values`, `items`, `update`, `pop`, and `setdefault`. Dictionary comprehensions help transform mappings. API responses, metadata, configuration, and model outputs often use dictionaries.

Common mistakes

A common mistake is accessing a missing key directly and getting `KeyError`. Use `get` when a missing key is acceptable. Do not mutate dictionaries while iterating over them without care.

Interview answer style

Interview answer: dict is mutable, insertion-ordered in modern Python, and provides fast key lookup using hashing.

Small code example

```
student = {"name": "Aman", "course": "Python"}
print(student.get("age", "not provided"))
```

Interview and practice focus

Be ready to explain dictionaries and json-style data in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

10. Operators and expressions

Concept

Python supports arithmetic, comparison, logical, assignment, identity, and membership operators. Expressions combine values and operators to produce results.

Important details

Important operators include // for floor division, % for remainder, ** for power, and, or, not for logical flow, is for identity, and in for membership. Operator precedence decides evaluation order.

Common mistakes

Do not confuse is and ==. is checks whether two references point to the same object, while == checks value equality.

Interview answer style

Interview answer should include examples of identity vs equality and membership testing.

Small code example

```
a = [1, 2]
b = [1, 2]
print(a == b)
print(a is b)
```

Interview and practice focus

Be ready to explain operators and expressions in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

11. Conditional statements

Concept

if, elif, and else allow code to make decisions. Conditions should be readable and simple. Complex conditions can be moved into helper functions.

Important details

Truthiness is useful but must be used carefully. For example, if items checks whether a list is non-empty. For None, prefer explicit checks using is None.

Common mistakes

Avoid deeply nested conditions. Use guard clauses when possible. Avoid using assignment when comparison is intended.

Interview answer style

Interview-style answer: conditionals control program flow based on boolean expressions and are used for validation, branching, and error prevention.

Small code example

```
marks = 76
if marks >= 90:
    grade = "A"
elif marks >= 60:
    grade = "B"
else:
    grade = "C"
```

Interview and practice focus

Be ready to explain conditional statements in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

12. for loops, while loops, break, and continue

Concept

Loops repeat code. A for loop iterates over items. A while loop runs until a condition becomes false. break exits a loop and continue skips the current iteration.

Important details

Use enumerate when index is needed and zip when iterating over two sequences together. while True is common for menu-driven programs and CLI chatbots, but it must include an exit condition.

Common mistakes

Avoid infinite loops. Avoid modifying the same collection you are iterating over unless you are using a safe approach.

Interview answer style

Explain when to use for vs while, then give a small example like reading questions until exit.

Small code example

```
while True:
    question = input("Ask: ")
    if question.lower() == "exit":
        break
    print(question)
```

Interview and practice focus

Be ready to explain for loops, while loops, break, and continue in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

13. Functions, parameters, and return values

Concept

Functions group reusable logic. They can accept positional arguments, keyword arguments, default arguments, *args, and **kwargs. return sends a value back to the caller.

Important details

Small functions are easier to test and debug. A function should usually do one clear task. In a RAG app, functions like clean_text, load_documents, create_chunks, and ask_question improve structure.

Common mistakes

Avoid mutable default arguments like def add_item(x, items=[]). Avoid functions that both print and return data unless needed.

Interview answer style

Interview answer: functions improve reusability, readability, modularity, and testing.

Small code example

```
def clean_text(text: str) -> str:
    return " ".join(text.split())

print(clean_text("hello  python"))
```

Interview and practice focus

Be ready to explain functions, parameters, and return values in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

14. Scope: local, global, nonlocal, and LEGB

Concept

Scope defines where a variable can be accessed. Python follows LEGB order: Local, Enclosing, Global, Built-in.

Important details

global allows modifying a global variable inside a function, while nonlocal works with enclosing function variables. In clean code, avoid unnecessary global state.

Common mistakes

A common mistake is reading and assigning a variable in a function without understanding local scope, causing UnboundLocalError.

Interview answer style

Interview-ready explanation: Python first searches variables in local scope, then enclosing scope, then global scope, then built-ins.

Small code example

```
x = 10
def show():
    x = 5
    print(x)
show()
print(x)
```

Interview and practice focus

Be ready to explain scope: local, global, nonlocal, and legb in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

15. Modules, packages, and imports

Concept

A module is a .py file. A package is a folder of modules. Imports allow code reuse across files. This is how clean projects separate responsibilities.

Important details

Absolute imports like `from src.loader import Loader` are common in structured projects. Avoid circular imports where two modules import each other.

Common mistakes

Do not name files the same as installed libraries, for example `streamlit.py` or `fastapi.py`, because imports may break.

Interview answer style

Explain modules using your RAG project: `document_loader.py` loads files, `chunking.py` splits text, `vector_store.py` creates or loads Chroma.

Small code example

```
from pathlib import Path
from src.document_loader import DocumentLoader
```

Interview and practice focus

Be ready to explain modules, packages, and imports in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

16. File handling and pathlib

Concept

File handling reads and writes data. pathlib provides a modern object-oriented way to work with paths. It is better than manual string path building.

Important details

Use with `open(...)` so files close automatically. Use `encoding="utf-8"` for text files. `Path.exists`, `Path.mkdir`, `Path.glob`, and `Path.rglob` are important for file-based projects.

Common mistakes

Common mistakes include wrong relative path, missing encoding, forgetting to close files, and expecting a deployment server to have local files that were not pushed.

Interview answer style

Interview answer: pathlib makes path handling cross-platform and readable.

Small code example

```
from pathlib import Path
DATA_DIR = Path("data")
for file in DATA_DIR.rglob("*.txt"):
    print(file.read_text(encoding="utf-8"))
```

Interview and practice focus

Be ready to explain file handling and pathlib in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

17. Exception handling

Concept

Exceptions represent runtime errors. try and except allow graceful recovery. else runs when no exception occurs. finally always runs and is used for cleanup.

Important details

Catch specific exceptions such as FileNotFoundError, ValueError, or KeyError. Logging the error helps debugging. In production, avoid exposing internal errors directly to users.

Common mistakes

Do not use bare except unless there is a very specific reason. Do not silently pass errors because it hides bugs.

Interview answer style

Interview answer: exception handling makes programs robust and prevents crashes from expected error cases.

Small code example

```
try:
    value = int("abc")
except ValueError as e:
    print("Invalid conversion", e)
```

Interview and practice focus

Be ready to explain exception handling in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

18. Comprehensions

Concept

Comprehensions provide concise syntax for creating lists, dictionaries, and sets. They are best for simple transformations and filters.

Important details

List comprehensions are often faster and cleaner than appending in a small loop. For complex logic, a normal loop is more readable.

Common mistakes

Do not write very long nested comprehensions that are hard to understand. Interviewers value readability.

Interview answer style

Explain with examples: squares, filtering even numbers, and creating a dictionary from a list.

Small code example

```
nums = [1, 2, 3, 4]
squares = [n*n for n in nums]
even = [n for n in nums if n % 2 == 0]
```

Interview and practice focus

Be ready to explain comprehensions in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

19. Lambda, map, filter, and sorted

Concept

lambda creates a small anonymous function. map applies a function to each item. filter selects items. sorted returns a new sorted sequence.

Important details

These tools are useful, but Python developers often prefer comprehensions for readability. key functions are commonly used with sorted.

Common mistakes

Do not overuse lambda when a named function is clearer. Do not confuse list.sort() with sorted(); sort changes the list and returns None.

Interview answer style

Interview answer should compare map/filter with comprehensions and mention sorted key.

Small code example

```
items = [{"name": "A", "score": 80}, {"name": "B", "score": 95}]
items_sorted = sorted(items, key=lambda x: x["score"], reverse=True)
```

Interview and practice focus

Be ready to explain lambda, map, filter, and sorted in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

20. Object references, mutability, and copying

Concept

Python variables reference objects. Some objects are mutable, like lists and dictionaries. Others are immutable, like strings, integers, and tuples.

Important details

Shallow copy copies the outer object but not nested objects. Deep copy copies nested objects too. Understanding this prevents bugs in lists of lists and configuration objects.

Common mistakes

A common mistake is using list multiplication for nested lists: `[[0]*3]*3` creates shared inner lists.

Interview answer style

Interview answer: assignment copies references, not necessarily objects; use copy methods when independent objects are needed.

Small code example

```
import copy
a = [[1], [2]]
b = copy.deepcopy(a)
b[0].append(99)
print(a, b)
```

Interview and practice focus

Be ready to explain object references, mutability, and copying in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

21. Basic standard library modules

Concept

Python standard library contains ready-to-use modules. Common modules include os, pathlib, json, datetime, math, random, re, collections, itertools, functools, logging, and typing.

Important details

json is used for API data, datetime for dates, re for regex, collections for specialized containers, logging for production messages, and typing for clarity.

Common mistakes

Do not install packages for tasks already solved by the standard library. Do not use random for security tokens; use secrets module.

Interview answer style

Interview answer: Python is called batteries included because the standard library covers many common tasks.

Small code example

```
import json
data = {"name": "Aman"}
text = json.dumps(data)
print(json.loads(text))
```

Interview and practice focus

Be ready to explain basic standard library modules in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

22. Regular expressions basics

Concept

Regular expressions search and manipulate text patterns. The `re` module supports matching, searching, replacing, splitting, and extracting groups.

Important details

Regex is useful for cleaning text, extracting emails, removing extra spaces, and validating simple patterns. For complex parsing, use proper parsers instead of huge regex.

Common mistakes

Common mistakes include not using raw strings `r"..."`, writing greedy patterns accidentally, and making unreadable expressions.

Interview answer style

Interview answer: regex is powerful for pattern matching but should be used carefully for maintainability.

Small code example

```
import re
text = "Aman  learns Python"
clean = re.sub(r"\s+", " ", text)
print(clean)
```

Interview and practice focus

Be ready to explain regular expressions basics in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

23. Reading user input and CLI programs

Concept

`input()` reads text from the user. CLI programs often run in a loop, accept commands, validate input, and print output.

Important details

For your normal non-Streamlit RAG app, a while loop can repeatedly read questions until the user types exit. Always handle empty questions gracefully.

Common mistakes

Do not assume input is already the correct type. Convert and validate safely.

Interview answer style

Interview answer: CLI apps are useful for quick testing before building a web UI.

Small code example

```
while True:
    q = input("Question: ").strip()
    if q.lower() == "exit":
        break
    if not q:
        print("Enter a question")
```

Interview and practice focus

Be ready to explain reading user input and cli programs in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

24. Basic RAG pipeline using Python concepts

Concept

A RAG pipeline loads documents, cleans text, splits into chunks, creates embeddings, stores embeddings in a vector database, retrieves relevant chunks, and generates an answer using an LLM.

Important details

Python basics support each step: pathlib for files, lists for documents, classes for pipeline components, functions for processing, dictionaries for metadata, and exception handling for reliability.

Common mistakes

Common RAG mistakes include recreating embeddings for every question, not caching the vector database, using very large chunks, and not instructing the model to answer only from context.

Interview answer style

Interview answer: RAG improves factual grounding by giving the LLM relevant external context before generation.

Small code example

```
docs = loader.load_all_documents()
cleaned = cleaner.clean_documents(docs)
chunks = chunker.chunk(cleaned)
vector_db = store.create_vector_store(model, chunks)
```

Interview and practice focus

Be ready to explain basic rag pipeline using python concepts in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.